北京邮电大学 本科毕业设计（论文）初期进度报告

Project Early-term Progress Report

学院 **School**	International School	专业 **Programme**	**Telecommunications Engineering with Management**		
姓 **Family name**	Li	名 **First Name**	Xiheng		
BUPT 学号 **BUPT number**	2022213192	**QM 学号** **QM number**	221166471	班级 **Class**	2022215107
论文题目 **Project Title**	From Traditional Operating Systems to AI Agent Operating Systems: Exploring AI-Native OS through the Lens of the MCP Protocol				

1. Introduction

The integration of Large Language Models (LLMs) into operating systems has catalyzed a paradigm shift from static instruction execution to dynamic, autonomous agent behavior. However, current frameworks operate atop general-purpose operating systems that lack native abstractions for managing the lifecycle of LLM inference. Relying solely on traditional process scheduling results in uncontrolled contention for the limited context window and lacks isolation mechanisms to prevent agents from monopolizing system resources or accessing unauthorized memory regions.

This project proposes an AI-Native Operating System architecture that integrates the Model Context Protocol (MCP) [1] directly into the kernel subsystem. By elevating "Agent Context" and "Tools" to first-class kernel citizens, we aim to replace passive user-space management with active kernel-level scheduling and security enforcement, thereby bridging the gap between traditional OS resource management and autonomous agent workflows.

2. Literature Review

2.1 Limitations of User-Space Agent Frameworks

Current agent architectures, such as AutoGPT [2] and Generative Agents [3], operate as user-space processes atop general-purpose operating systems. However, these traditional OS environments lack native abstractions for LLM-centric workflows. Qin et al. [4] pointed out that existing frameworks lack a unified mechanism to manage LLM resource contention. Instead of scheduled access, agents employ an inefficient "trial-and-error" approach: they

attempt to load prompts into GPU memory until hitting hardware limits (e.g., CUDA out-of-memory exceptions), forcing expensive deallocation and multiple retry attempts that severely degrade system throughput.

Furthermore, current general-purpose OS abstractions treat Agent Context as opaque data blobs, forcing frameworks to manage state retrieval sequentially in user-space, ignoring potential data-level parallelism. Lacking native parallel primitives for "Agent Context," the system fails to leverage multi-core parallelism for concurrent reasoning tasks, forcing agents to process information strictly linearly.

2.2 The "LLM as OS" Metaphor vs. Reality

Recent works like AIOS [5] , MemGPT[6] , and Xserve [7] have popularized the concept of an "OS for LLMs." MemGPT introduces a virtual memory hierarchy to manage limited context windows, Xserve introduces a system-level abstraction and service, which utilizes a virtual capability layer with a unified API to streamline the integration of AI into applications, while AIOS proposes an LLM-centric scheduler that isolates resources and services into an AIOS kernel to improve scheduling, context management, and resource utilization efficiency. However, a critical analysis reveals fundamental architectural limitations in these systems, the MemGPT focus mainly on context memory and text generation, which is not enough for an AI agent operating system. While AIOS demonstrates improved throughput across various agent benchmarks, it operates entirely as a user-space application (running on the Python interpreter). Consequently, it lacks strict isolation: its 'Access Manager' relies on high-level logic checks, meaning a malicious agent could potentially crash the host process or bypass restrictions. Furthermore, AIOS relies on software-level context interruption rather than true hardware preemptive scheduling of GPU resources, preventing it from enforcing hard real-time guarantees for mission-critical agent responses.

Also, in the security level, the existing AI operating systems lack isolation. A malicious agent can easily crash the host Python process or access unauthorized memory, as demonstrated in recent prompt-injection attacks on agentic systems [8][9].

Some mobile operating systems have begun integrating machine learning workloads directly into the system infrastructure beyond simple application-level and configured out some solutions to tackle the above problems.

In 2024, Apple introduced a system-wide intelligence layer deeply integrated into iOS and macOS[10]. A key component of its architecture is the combination of App Intents and the Semantic Index. App Intents is a Swift-based protocol that redefines application functionalities using a standardized intent schema and exposes them to the operating system, enabling the LLM to invoke applications to fulfill user instructions. The Semantic Index organizes user data across applications to provide context-aware capabilities. Apple adopts a hybrid execution model, processing sensitive tasks on device while offloading more computationally intensive tasks to Private Cloud Compute.

Google introduced Private Compute Core to bring a new layer of privacy to the Android ecosystem. for OS-level ML features [11]. Private Compute Core is an open source, secure environment that is isolated from the rest of the operating system and apps. It ensures that sensitive data processed by ML models stays isolated from the network via strict API boundaries, communicating with the cloud only through Private Compute Services for model updates [12].

While these industrial solutions validate the necessity of OS-level AI integration, they are proprietary and siloed ecosystems. They lack a unified, open interface for third-party agents to interact with the kernel's context management. This project differs by proposing an open-source, kernel-native implementation using the standardized MCP, aiming to democratize system-level agent capabilities for the Linux ecosystem.

2.3 Kernel-Level Opportunities and the MCP Protocol

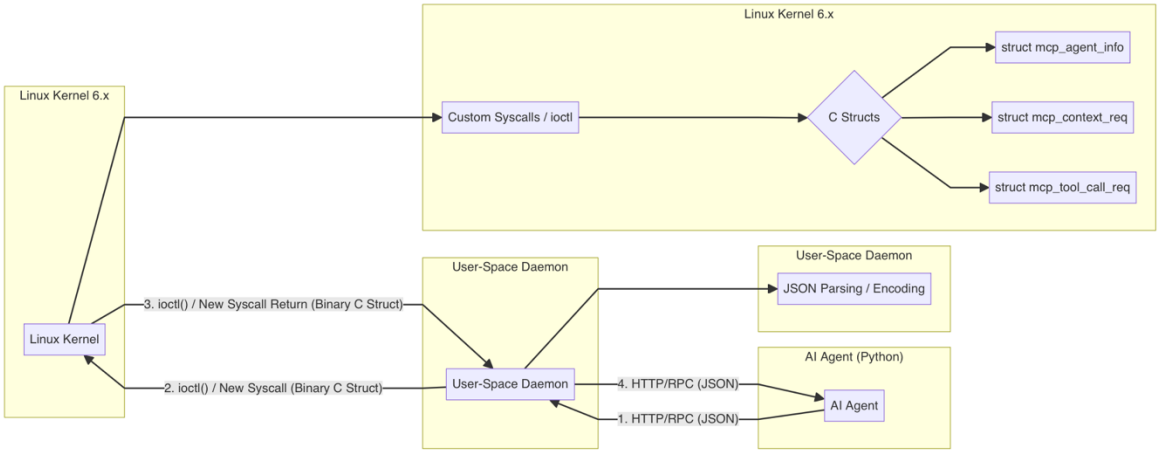
The MCP provides a standardized schema for linking agents to data, and it can effectively prevent the malicious instructions sent by agents. However, current MCP implementations are exclusively high-level (TypeScript/Python). There is a distinct lack of research on implementing MCP primitives as System Calls. This project fills this gap by exploring a Kernel-Level MCP Module to offer lower latency and mandatory access control.

3. Project Progress Summary

3.1 Completed Work

In accordance with Task 1 and the early stages of Task 2 in the project work plan, the following progress has been made:

- Theoretical Framework Analysis: I have completed a comparative analysis between traditional process-centric paradigms (Process Control Blocks - PCB) and agent-centric demands (Agent Context Blocks). I determined that the Linux task_struct needs to be extended or pointed to by a new auxiliary structure to hold MCP metadata.
- MCP Protocol Analysis: I have deeply analyzed the MCP specification and security, specifically the lifecycle of initialization and the prompts, resources, and tools primitives. I have drafted a mapping table that translates MCP JSON-RPC messages into theoretical C-language kernel structures.



- Environment Setup: A development environment has been successfully established on my MacBook using VMware, the Linux version is Ubuntu 22.04 LTS.
    - Compiled Linux Kernel 6.x source code to ensure the environment is ready for custom module insertion.
    - Installed and tested FastMCP to understand the standard user-space behavior of the MCP protocol.
    - Configured QEMU/KVM for safe kernel debugging (GDB) to prevent crashing the host machine during development.

In the future, I will deploy the system on a high-performance server to evaluate scalability and accurate performance metrics under multi-agent workloads

3.2 Comparison with Work Plan

The project is currently on schedule.

- Planned: Analyze OS architectures and MCP internal structure by mid-January.
- Actual: The literature review and architectural analysis are complete. The mapping of MCP functions to kernel system calls (Task 2) is currently in progress.
- Adjustment: The complexity of parsing JSON (used by MCP) inside the kernel was identified as a technical risk. I have decided to implement a lightweight binary serialization protocol for the kernel boundary, with a user-space daemon handling the JSON conversion, which slightly refines the approach in Task 3.

4. References

[1]. Model Context Protocol, "Model Context Protocol Specification," 2024. [Online]. Available: https://spec.modelcontextprotocol.io/

[2]. T. Richards, "Auto-GPT: An Autonomous GPT-4 Experiment," GitHub Repository, 2023. [Online]. Available: https://github.com/Significant-Gravitas/Auto-GPT

[3]. J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative Agents: Interactive Simulacra of Human Behavior," in Proc. ACM Symp. User Interface Softw. Technol. (UIST), 2023.

[4]. Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. Toolllm: Facilitating large language models to master 16000+ real-world apis. ICLR, 2024.

[5]. Kai Mei et al., "AIOS: LLM Agent Operating System," arXiv preprint arXiv:2403.16971, 2024.

[6]. C. Packer, V. Fang, S. G. Patil, K. Lin, S. Wooders, and J. Gonzalez, "MemGPT: Towards LLMs as Operating Systems," arXiv preprint arXiv:2310.08560, 2024.

[7]. Jinrong Yang, Zimeng Wang, Rong Chen, and Haibo Chen. 2025. A System-level Abstraction and Service for Flourishing AI-powered Applications. In Proceedings of the 16th ACM SIGOPS Asia-Pacific Workshop on Systems (APSys '25). Association

for Computing Machinery, New York, NY, USA, 106–114. https://doi.org/10.1145/3725783.3764406

[8]. OWASP GenAI Security Project, "LLM01: Prompt Injection," OWASP Top 10 for Large Language Model Applications, 2025. [Online]. Available: https://genai.owasp.org/llmrisk/llm01-prompt-injection/

[9]. F. Liu et al., "Prompt Injection attack against LLM-integrated Applications," arXiv preprint arXiv:2306.05499, 2023.

[10]. Apple Inc., "Apple Intelligence: System Architecture and Privacy," Apple Platform Security Guide, 2024. [Online]. Available: https://security.apple.com/documentation/

[11]. Google. Android Private Compute Core: Technical Whitepaper. Google Open Source Security, 2022. [Online]. Available: https://services.google.com/fh/files/misc/android-security-paper-2024.pdf

[12]. Google. Private Compute Services. GitHub Repository, 2022. [Online]. Available: https://github.com/google/private-compute-services

是否符合进度？On schedule as per GANTT chart?

YES

下一步 Next steps:

Kernel Data Structure Definition: Define the struct mcp_agent and struct mcp_context in C code within the kernel module to store agent metadata.

Kernel Module Skeleton: Write the initial Character Device Driver (/dev/mcp_core) that allows user-space processes to "register" as agents via ioctl calls.

Prototype Integration: Develop a minimal user-space Python script using ctypes to interact with the kernel module, demonstrating basic agent registration and context retrieval.